

CSS

Cascading Style Sheets

Beginner notes for students who just finished HTML

You already know how to build the **skeleton** of a page with HTML. CSS is how you give it **skin, colour, and clothes**. HTML decides **what** is on the page; CSS decides **how it looks**. This guide takes you from your very first line of CSS to building real layouts with Flexbox.

1. What Is CSS and Why Do We Need It?

Imagine HTML as a house with no paint, no furniture arrangement, and no decoration — just walls and rooms. It works, but it looks plain. **CSS (Cascading Style Sheets)** is the language we use to decorate that house: choosing colours, spacing things out, picking fonts, and arranging where everything sits.

The word “**cascading**” matters. It means styles flow down and can override each other based on rules. Don't worry about that yet — we'll come back to it. For now, just know CSS controls appearance.

In plain English

HTML = structure (the content). CSS = presentation (the look). Keeping them separate makes your code easier to read and change.

2. Three Ways to Add CSS

There are three places you can write CSS. You should understand all three, but one is the clear winner for real projects.

a) Inline CSS — directly on an element

You add a **style** attribute right on the HTML tag.

```
<p style="color: blue; font-size: 18px;">Hello</p>
```

Avoid this for real work. It mixes structure and style, and you'd have to repeat it on every element. Useful only for quick tests.

b) Internal CSS — in a <style> tag

You put a `<style>` block inside the `<head>` of your HTML file.

```
<head>
  <style>
    p {
      color: blue;
      font-size: 18px;
    }
  </style>
</head>
```

Fine for a single small page, but the styles only work on that one file.

c) External CSS — a separate .css file (recommended)

You write your CSS in its own file, for example `styles.css`, and link it from the HTML.

```
<!-- inside the <head> of index.html -->
<link rel="stylesheet" href="styles.css">
/* styles.css */
p {
  color: blue;
  font-size: 18px;
}
```

Best practice

Always use external CSS for real projects. One stylesheet can style your whole website, and changing one line updates every page. This is how professionals work.

3. CSS Syntax: The Building Block

Every CSS instruction follows the exact same pattern. Learn this shape once and all CSS makes sense.

```
selector {
  property: value;
}
```

Let's break a real example down piece by piece:

```
h1 {
  color: crimson;
  font-size: 32px;
}
```

- `h1` — the **selector**. It chooses **which** elements to style (here, all `<h1>` headings).
- `color` — a **property**. The thing you want to change.
- `crimson` — the **value**. What you want to change it to.
- The `{ }` curly braces hold the rules; each line ends with a **semicolon (;)**.

Common beginner mistake

Forgetting the semicolon at the end of a line, or the closing curly brace. If your styles suddenly stop working, check these two things first.

4. Selectors: Choosing What to Style

Selectors are how you point CSS at the right elements. These are the ones you'll use every single day.

Selector	Targets	Example
Element	All tags of that type	<code>p { }</code> styles every paragraph
Class (.)	Any element with that class	<code>.btn { }</code> styles <code>class="btn"</code>
ID (#)	The one element with that id	<code>#header { }</code> styles <code>id="header"</code>
Descendant	Elements inside another	<code>nav a { }</code> styles links in nav
Universal (*)	Everything on the page	<code>* { }</code> styles all elements

Classes vs IDs — the key rule

- **Class (.)** — reusable. Many elements can share the same class. Use these most of the time.
- **ID (#)** — unique. Only **one** element per page should have a given id. Use sparingly.

```
<button class="btn">Save</button>
<button class="btn">Cancel</button>

/* both buttons get styled by one rule */
.btn {
  background: purple;
  color: white;
}
```

5. Colours, Text and Common Units

Ways to write colours

Method	Example	Notes
Name	<code>color: tomato;</code>	~140 built-in names; easy but limited
HEX	<code>color: #5B2A86;</code>	Most common; # plus 6 digits
RGB	<code>color: rgb(91,42,134);</code>	Red, Green, Blue (0–255)
RGBA	<code>rgba(0,0,0,0.5)</code>	Same plus alpha (transparency)

Styling text

```
p {
  color: #333333;          /* text colour */
  font-family: Arial, sans-serif;
  font-size: 16px;         /* size of text */
  font-weight: bold;        /* bold / normal / 700 */
  text-align: center;       /* left / right / center */
  line-height: 1.6;        /* spacing between lines */
}
```

Units you'll actually use

Unit	Meaning	When to use
px	Pixels — a fixed size	Borders, small precise sizes
%	Percent of the parent	Widths that should scale
em	Relative to current font size	Spacing tied to text size
rem	Relative to the root font size	Consistent sizing (preferred)

6. The Box Model — The Most Important Idea

This is the concept that confuses beginners the most, so read it slowly. **Every element on a page is a rectangular box**, even text and images. Each box has four layers, from the inside out:

1. **Content** — the actual text or image inside the box.
2. **Padding** — space **inside** the box, between the content and the border.
3. **Border** — the line drawn around the padding.
4. **Margin** — space **outside** the box, pushing other elements away.

Easy way to remember

Think of a framed photo. The photo is the content. The white mat around it is the padding. The frame is the border. The gap between this frame and the next frame on the wall is the margin.

```
.card {
  width: 300px;
  padding: 20px;    /* space inside */
  border: 2px solid #5B2A86;
  margin: 16px;     /* space outside */
}
```

The box-sizing lifesaver

By default, padding and border get **added on top of** the width, making boxes bigger than expected. Add this one rule at the top of every stylesheet and that headache disappears:

```
* {
  box-sizing: border-box;
}
```

Now `width` means the **total** width, padding and border included. Much more intuitive.

7. Display: Block vs Inline

Elements behave in one of two main ways by default, and knowing the difference explains a lot of “why won’t this sit next to that?” moments.

Type	Behaviour	Examples
<code>block</code>	Takes full width, starts on a new line	<code>div</code> , <code>p</code> , <code>h1</code> , <code>section</code>
<code>inline</code>	Only as wide as its content, sits in a line	<code>span</code> , <code>a</code> , <code>strong</code>
<code>inline-block</code>	Flows in a line but accepts width/height	buttons, nav items
<code>none</code>	Hides the element completely	toggles, menus

8. Flexbox: Modern Layout Made Simple

Before Flexbox, arranging boxes side by side was genuinely painful. Flexbox makes it easy. You turn an element into a **flex container**, and its direct children become **flex items** you can line up and space out.

```
.container {
  display: flex;           /* turn on flexbox */
  justify-content: center; /* horizontal alignment */
  align-items: center;     /* vertical alignment */
  gap: 16px;              /* space between items */
}
```

The two properties you’ll use constantly

- `justify-content` — positions items along the **main (horizontal) axis**. Try `flex-start`, `center`, `space-between`.
- `align-items` — positions items along the **cross (vertical) axis**. Try `center`, `flex-start`, `stretch`.

The famous trick

`display: flex; justify-content: center; align-items: center;` — these three lines perfectly centre anything inside a box, both horizontally and vertically. It used to be one of the hardest things in CSS.

9. The Cascade and Specificity

Remember the “cascading” in CSS? When two rules try to style the same element, CSS needs to decide who wins. Two ideas settle it:

1. Specificity — more specific selectors win

From weakest to strongest: element selectors < classes < IDs < inline styles. An ID beats a class; a class beats a plain element.

```
p      { color: black; } /* weakest */
.note  { color: blue;  } /* beats element */
#special { color: red;  } /* beats class */
```

2. Source order — last one wins (on a tie)

If two rules have the **same** specificity, the one written **later** in the file wins. This is why the order of your CSS matters.

Avoid !important

You'll discover !important forces a rule to win no matter what. It feels handy but creates messy code that's hard to fix later. Reach for better selectors instead, and save !important for genuine emergencies.

10. Quick Reference Cheat-Sheet

Keep this handy while you practise.

Property	What it does	Example value
color	Text colour	#333, red
background	Background colour/image	#f4f1f8
font-size	Size of text	16px, 1.2rem
padding	Space inside the box	20px
margin	Space outside the box	16px auto
border	Line around the box	1px solid #ccc
width / height	Box dimensions	300px, 50%
display	Layout behaviour	flex, block, none
text-align	Horizontal text position	center
border-radius	Rounded corners	8px, 50%

11. Practice: Build a Profile Card

Theory sticks when you build something. Try this small challenge using only what's in these notes.

Your task: Create a centred “profile card” with a name, a short bio, and a button. Aim for:

1. A card with padding, a border, and rounded corners.
2. A heading and a paragraph inside it.
3. A styled button with a background colour and white text.

4. The whole card centred on the page using Flexbox.

```
<!-- index.html -->
<div class="page">
  <div class="card">
    <h2>Jane Doe</h2>
    <p>Junior web developer learning CSS.</p>
    <button class="btn">Follow</button>
  </div>
</div>
/* styles.css - try to write this yourself first! */
* { box-sizing: border-box; margin: 0; }

.page {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  background: #f4f1f8;
}

.card {
  width: 300px;
  padding: 24px;
  background: white;
  border-radius: 12px;
  border: 1px solid #ddd;
  text-align: center;
}

.btn {
  margin-top: 12px;
  padding: 10px 20px;
  background: #5B2A86;
  color: white;
  border: none;
  border-radius: 6px;
}
```

Next step after CSS

Once you're comfortable here, look into CSS Grid (for two-dimensional layouts) and media queries (for making sites work on phones). But master the box model and Flexbox first — they carry you a very long way.

— End of Notes —